

1. File Descriptors and Redirections

A file descriptor (**FD**) in Unix/Linux operating systems is a reference, maintained by the kernel, that allows the system to manage Input/Output (**I/O**) operations. It acts as a unique identifier for an open file, socket, or any other I/O resource. In Windows-based operating systems, this is known as a file handle. Essentially, the file descriptor is the system's way of keeping track of active **I/O** connections, such as reading from or writing to a file.

Think of it as a ticket number you get when checking in your coat at a coatroom. The ticket (file descriptor) represents your connection to your coat (file or resource), and whenever you need to retrieve your coat (perform I/O), you present the ticket to the attendant (operating system) who knows exactly where your coat is stored (which resource the file descriptor refers to). Without the ticket, you'd have no way of efficiently accessing your coat among the many others stored, just as without a file descriptor, the operating system wouldn't know which resource to interact with. You will soon see why file descriptors are so important and why understanding them is crucial as we dive into the upcoming examples.

By default, the first three file descriptors in Linux are:

1. Data Stream for Input
 - **STDIN** - 0
2. Data Stream for Output
 - **STDOUT** - 1
3. Data Stream for Output that relates to an error occurring.
 - **STDERR** - 2



4. STDIN and STDOUT

Let us see an example with **cat**. When running **cat**, we give the running program our standard input (**STDIN** - **FD 0**), marked **green**, wherein this case "SOME INPUT" is. As soon as we have confirmed our input with **[ENTER]**, it is returned to the terminal as standard output (**STDOUT** - **FD 1**), marked **red**.

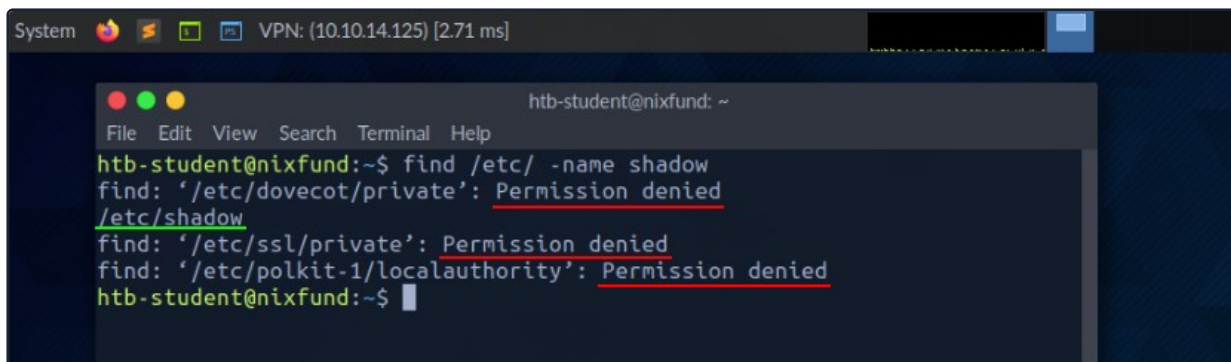
```
System  VPN: (10.10.14.125) [2.48 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ cat
Think Outside The Box
Think Outside The Box
```

The screenshot shows a terminal window with a dark background. The prompt is `htb-student@nixfund: ~`. The user has entered `cat`. The output shows two lines of text: `Think Outside The Box` and `Think Outside The Box`. The first line is highlighted with a green box, and the second line is highlighted with a red box. A red arrow points to the second line.

4. STDOUT and STDERR

In the next example, by using the **find** command, we will see the standard output (**STDOUT** - **FD 1**) marked in **green** and standard error (**STDERR** - **FD 2**) marked in red.

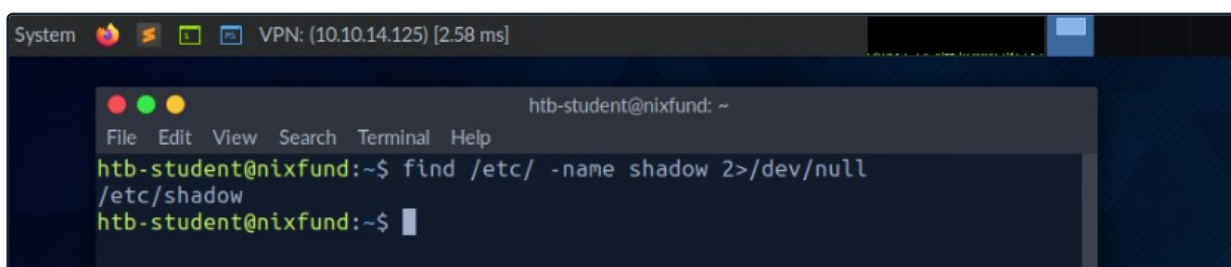
File Descriptors and Redirections
<pre>dbcypth0n@htb[/htb]\$ find /etc/ -name shadow</pre>



```
System  VPN: (10.10.14.125) [2.71 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ find /etc/ -name shadow
find: '/etc/dovecot/private': Permission denied
/etc/shadow
find: '/etc/ssl/private': Permission denied
find: '/etc/polkit-1/localauthority': Permission denied
htb-student@nixfund:~$
```

In this case, the error is marked and displayed with "Permission denied". We can check this by redirecting the file descriptor for the errors (FD 2 - STDERR) to "/dev/null." This way, we redirect the resulting errors to the "null device," which discards all data.

File Descriptors and Redirections
dbcypth0n@htb[/htb]\$ find /etc/ -name shadow 2>/dev/null

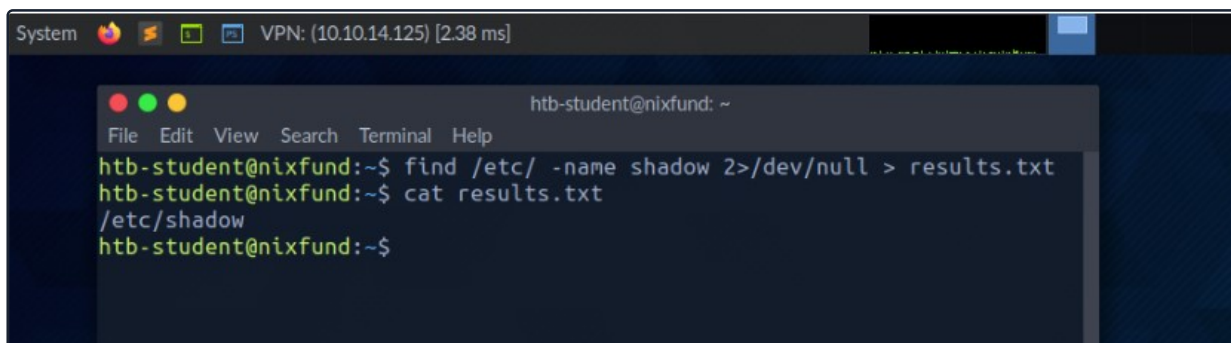


```
System  VPN: (10.10.14.125) [2.58 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ find /etc/ -name shadow 2>/dev/null
/etc/shadow
htb-student@nixfund:~$
```

4. Redirect STDOUT to a File

Now we can see that all errors (STDERR) previously presented with "Permission denied" are no longer displayed. The only result we see now is the standard output (STDOUT), which we can also redirect to a file with the name `results.txt` that will only contain standard output without the standard errors.

File Descriptors and Redirections
dbcypth0n@htb[/htb]\$ find /etc/ -name shadow 2>/dev/null > results.txt



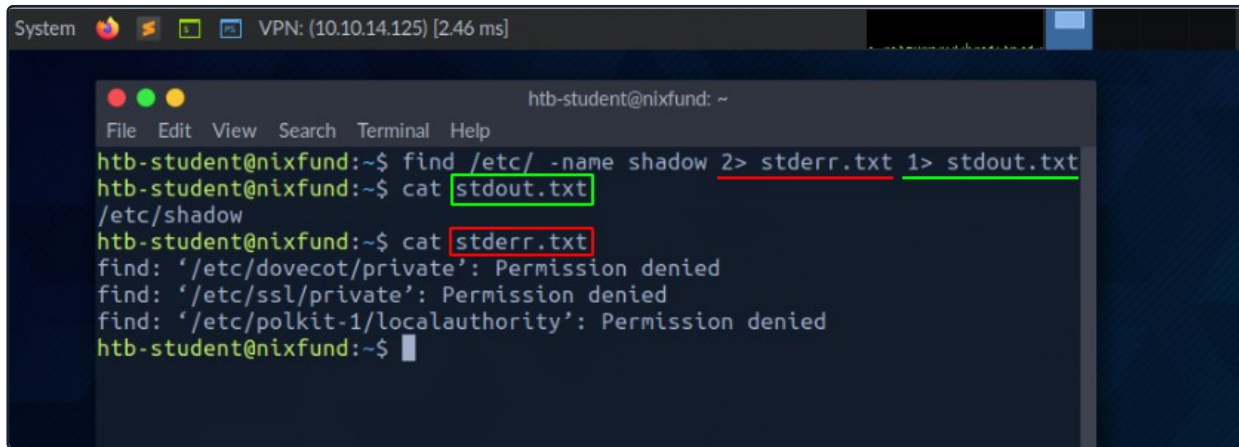
```
System  VPN: (10.10.14.125) [2.38 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ find /etc/ -name shadow 2>/dev/null > results.txt
htb-student@nixfund:~$ cat results.txt
/etc/shadow
htb-student@nixfund:~$
```

4. Redirect STDOUT and STDERR to Separate Files

We should have noticed that we did not use a number before the greater-than sign (>) in the last example. That is because we redirected all the standard errors to the "null device" before, and the only output we get is the standard output (FD 1 - STDOUT). To make this more precise, we will redirect standard error (FD 2 - STDERR) and standard output (FD 1 - STDOUT) to different files.

File Descriptors and Redirections

```
dbcyph0n@htb[/htb]$ find /etc/ -name shadow 2> stderr.txt 1> stdout.txt
```



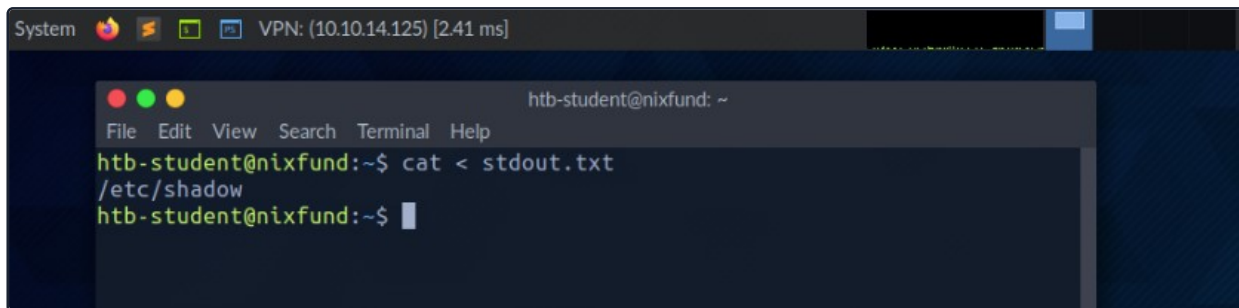
```
System [2.46 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ find /etc/ -name shadow 2> stderr.txt 1> stdout.txt
htb-student@nixfund:~$ cat stdout.txt
/etc/shadow
htb-student@nixfund:~$ cat stderr.txt
find: '/etc/dovecot/private': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/polkit-1/localauthority': Permission denied
htb-student@nixfund:~$
```

4. Redirect STDIN

As we have already seen, in combination with the file descriptors, we can redirect errors and output with greater-than character (>). This also works with the lower-than sign (<). However, the lower-than sign serves as standard input (FD 0 - STDIN). These characters can be seen as "direction" in the form of an arrow that tells us "from where" and "where to" the data should be redirected. We use the `cat` command to use the contents of the file "stdout.txt" as STDIN.

File Descriptors and Redirections

```
dbcyph0n@htb[/htb]$ cat < stdout.txt
```



```
System [2.41 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ cat < stdout.txt
/etc/shadow
htb-student@nixfund:~$
```

4. Redirect STDOUT and Append to a File

When we use the greater-than sign (>) to redirect our `STDOUT`, a new file is automatically created if it does not already exist. If this file exists, it will be overwritten without asking for confirmation. If we want to append `STDOUT` to our existing file, we can use the double greater-than sign (>>).

File Descriptors and Redirections

```
dbcyph0n@htb[/htb]$ find /etc/ -name passwd >> stdout.txt 2>/dev/null
```



```
System [2.31 ms]
```

```
htb-student@nixfund: ~  
File Edit View Search Terminal Help  
htb-student@nixfund:~$ find /etc/ -name passwd >> stdout.txt 2>/dev/null  
htb-student@nixfund:~$  
htb-student@nixfund:~$ cat stdout.txt  
/etc/pam.d/passwd  
/etc/cron.daily/passwd  
/etc/passwd  
htb-student@nixfund:~$
```

4. Redirect STDIN Stream to a File

We can also use the double lower-than characters (<<) to add our standard input through a stream. We can use the so-called **End-Of-File** (EOF) function of a Linux system file, which defines the input's end. In the next example, we will use the **cat** command to read our streaming input through the stream and direct it to a file called "stream.txt."

File Descriptors and Redirections

```
dbcypth0n@htb[/htb]$ cat << EOF > stream.txt
```

```
System VPN: (10.10.14.125) [2.37 ms]  
htb-student@nixfund: ~  
File Edit View Search Terminal Help  
htb-student@nixfund:~$ cat << EOF > stream.txt  
> Hack  
> The  
> Box  
> EOF  
htb-student@nixfund:~$ cat stream.txt  
Hack  
The  
Box  
htb-student@nixfund:~$
```

4. Pipes

Another way to redirect **STDOUT** is to use pipes (|). These are useful when we want to use the **STDOUT** from one program to be processed by another. One of the most commonly used tools is **grep**, which we will use in the next example. **Grep** is used to filter **STDOUT** according to the pattern we define. In the next example, we use the **find** command to search for all files in the "/etc/" directory with a ".conf" extension. Any errors are redirected to the "null device" (/dev/null). Using **grep**, we filter out the results and specify that only the lines containing the pattern "systemd" should be displayed.

File Descriptors and Redirections

```
dbcypth0n@htb[/htb]$ find /etc/ -name *.conf 2>/dev/null | grep systemd
```

```
System VPN: (10.10.14.125) [2.38 ms]  
htb-student@nixfund: ~  
File Edit View Search Terminal Help  
htb-student@nixfund:~$ find /etc/ -name *.conf 2>/dev/null | grep systemd  
/etc/systemd/system.conf  
/etc/systemd/timesyncd.conf  
/etc/systemd/journald.conf  
/etc/systemd/user.conf  
/etc/systemd/logind.conf  
/etc/systemd/resolved.conf
```

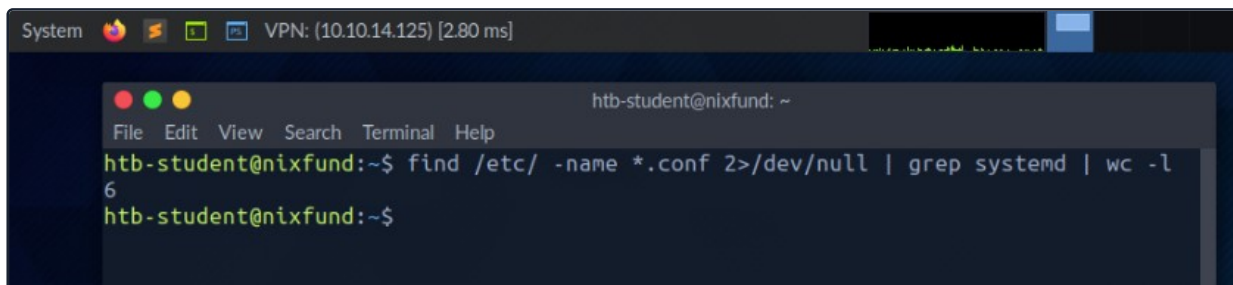


```
htb-student@nixfund:~$
```

The redirections work, not only once. We can use the obtained results to redirect them to another program. For the next example, we will use the tool called `wc`, which should count the total number of obtained results.

File Descriptors and Redirections

```
dbcyph0n@htb[/htb]$ find /etc/ -name *.conf 2>/dev/null | grep systemd | wc -l
```



```
System  VPN: (10.10.14.125) [2.80 ms]
htb-student@nixfund: ~
File Edit View Search Terminal Help
htb-student@nixfund:~$ find /etc/ -name *.conf 2>/dev/null | grep systemd | wc -l
6
htb-student@nixfund:~$
```

Now that we have a fundamental understanding of file descriptors, redirections, and pipes, we can structure our commands more efficiently to extract the exact information we need. This knowledge allows us to manipulate how input and output flows between files, processes, and the system, enabling us to handle data more effectively. By leveraging these tools, we can streamline tasks, avoid unnecessary steps, and work with files and system resources in a much more organized and efficient manner, ultimately enhancing our productivity and precision in managing operations.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

EU Academy 3

Low Load

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

67ms

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

4. Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): [Click here to spawn the target system!](#)

SSH to with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 1  How many files exist on the system that have the ".log" file extension?

Submit your answer here...

+10 Streak pts

 Submit

 Show Solution

+ 0  How many total packages are installed on the target system?

Submit your answer here...

+10 Streak pts

 Submit

 Show Solution

← Previous

Next →

 Cheat Sheet

? Go to Questions

5. Table of Contents

6. Introduction

Linux Structure



Linux Distributions



Introduction to Shell



6. The Shell

Prompt Description



Getting Help



 System Information



6. Workflow

 Navigation



 Working with Files and Directories



 Editing Files



 Find Files and Directories



 [File Descriptors and Redirections](#)

 Filter Contents

 Regular Expressions

 Permission Management

6. System Management

 User Management

 Package Management

 Service and Process Management

 Task Scheduling

 Network Services

 Working with Web Services

 Backup and Restore

 File System Management

 Containerization



6. Linux Networking

 Network Configuration

Remote Desktop Protocols in Linux

6. Linux Hardening

Linux Security

 Firewall Setup

 System Logs and Monitoring

6. Linux Distributions vs Solaris

Solaris

6. Tips & Tricks

Shortcuts

5. My Workstation

OFFLINE

Start Instance

∞ / 1 spawns left

